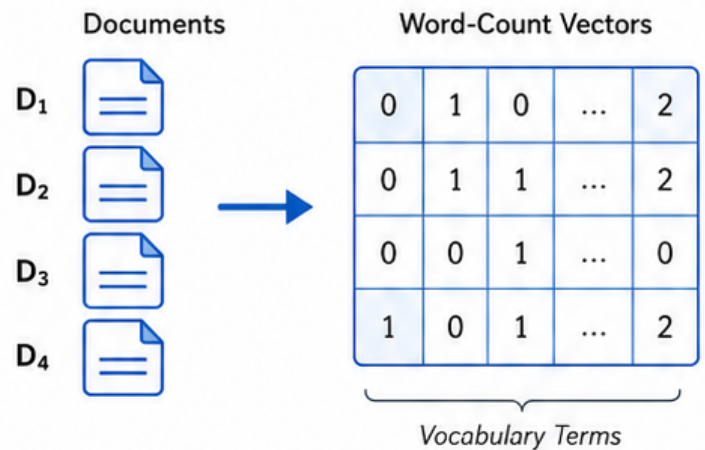


Representation of Text

AIML Study Notes

Common Text Representation Methods

- 1 Bag of Words
- 2 One-Hot Encoding
- 3 TF-IDF Encoding
- 4 N-gram Representation
- 5 Count Vectorizer



Example Corpus

D₁ = "The boy and the dog"

D₂ = "The cat and the dog"

D₃ = "Andrew is a tall boy"

D₄ = "Ram is a good boy. Ratna is also good"

1 Bag of Words / Count Vectorizer

A bag-of-words model converts each document into a vector based on word frequency, ignoring grammar and word order.



Tokenized Documents

D₁ → the, boy, and, the, dog

D₂ → the, cat, and, the, dog

D₃ → andrew, is, a, tall, boy

D₄ → ram, is, a, good, boy, ratna, is, also, good



Vocabulary / Corpus

[also, and, andrew, boy, cat, dog, good, is, ram, ratna, tall, the]

Vocabulary size = 12 tokens



Vector for D₁

Using the vocabulary order above:

D₁ = [0, 1, 0, 1, 0, 1, 0, 0, 0, 0, 0, 2]



Count Vector Example

Term	D ₁	D ₂	D ₃	D ₄
also	0	0	0	1
and	1	1	0	0
andrew	0	0	1	0
boy	1	0	1	1
cat	0	1	0	0
dog	1	1	0	0
good	0	0	0	2
is	0	0	1	2
ram	0	0	0	1
ratna	0	0	0	1
tall	0	0	1	0
the	2	2	0	0

Representation of Text

2. One-Hot Encoding

One-hot encoding represents each word as a binary vector in which only one position is 1 and all others are 0.



✔ **Example:** 'boy' = [0, 1, 0, 0, 0, 0]

Vocabulary	One-Hot Vectors					
	and	boy	cat	dog	is	the
and	1	0	0	0	0	0
boy	0	1	0	0	0	0
cat	0	0	1	0	0	0
dog	0	0	0	1	0	0
is	0	0	0	0	1	0
the	0	0	0	0	0	1

3. TF-IDF Encoding

TF-IDF highlights important words by combining term frequency (TF) and inverse document frequency (IDF). Common words get lower weight, while informative words get higher weight.

⚠ Stop words often removed:
the, and, is, also, a

Cleaned Documents

D ₁	→	boy, dog
D ₂	→	cat, dog
D ₃	→	andrew, tall, boy
D ₄	→	ram, good, ratna, good, boy

Cleaned Vocabulary

[andrew, boy, cat, dog, good, ram, ratna, tall]
Vocabulary size = 8

TF-IDF Formulas

$$TF-IDF(t, d) = TF(t, d) \times IDF(t)$$
$$IDF(t) = \log \left(\frac{1 + N}{1 + DF(t)} \right) + 1$$

$N = 4$ documents

Document Frequency and IDF

Word	DF	IDF
andrew	1	1.916
boy	3	1.223
cat	1	1.916
dog	2	1.511
good	1	1.916
ram	1	1.916
ratna	1	1.916
tall	1	1.916

Worked Example

For $D_1 = [boy, dog]$
Using vocabulary order:
[andrew, boy, cat, dog, good, ram, ratna, tall]

$$TF-IDF(D_1) = [0, 1.223, 0, 1.511, 0, 0, 0, 0]$$

Normalized vector ≈

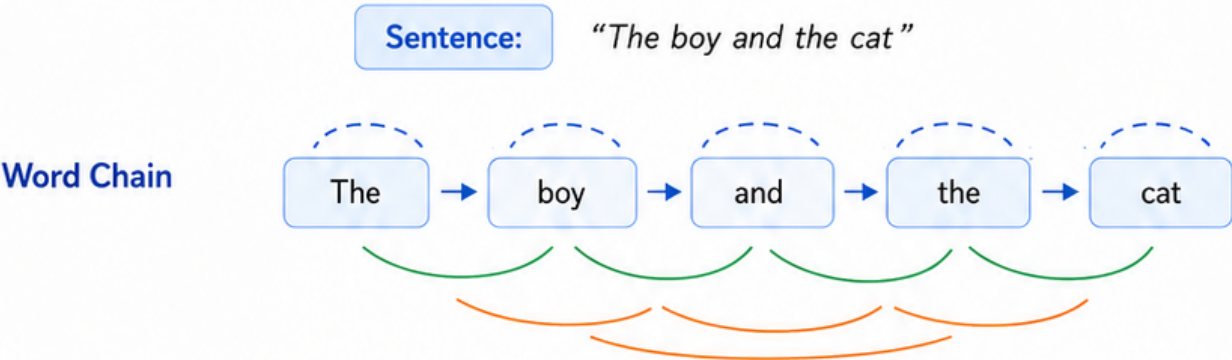
$$[0, 0.629, 0, 0.777, 0, 0, 0, 0]$$

Representation of Text

4 N-gram Representation

N-grams capture short sequences of words, helping preserve local word order that bag-of-words ignores.

Worked Example



- 1 Unigrams ($n = 1$) [the, boy, and, the, cat]
- 2 Bigrams ($n = 2$) [the boy, boy and, and the, the cat]
- 3 Trigrams ($n = 3$) [the boy and, boy and the, and the cat]

i When $n = 2$, we get **bigrams**. When $n = 3$, we get **trigrams**.

Quick Comparison of Methods

Method	Idea	Strength	Limitation
 Bag of Words	counts word frequency	simple and interpretable	ignores word order
 One-Hot Encoding	unique binary vector per word	easy representation	sparse and high-dimensional
 TF-IDF	frequency weighted by importance	highlights informative words	still ignores deep semantics
 N-gram	uses word sequences	captures local context	vocabulary grows quickly

Key Takeaways

-  Count-based methods are simple baselines.
-  One-hot vectors are sparse but easy to build.
-  TF-IDF reduces the impact of common words.
-  N-grams help preserve short phrase structure.

Representation of Text

4 Web Scraping



Web scraping is the process of automatically collecting data from websites.

Output Formats



CSV



Excel



JSON



API



Database

Web Scraping Workflow



Typical Example

Fields to extract	Example values
Product Name	iPhone 15
Price	₹79,000
Rating	4.5
Review Count	2740
Availability	In Stock

HTML Clues Used for Scraping

```
<h1>Product Name</h1>
<span class="price">₹999</span>
<p class="rating">4.5</p>
<table>
  <tr><td>...</td></tr>
</table>
```

Basic Steps

- 1 Choose website
- 2 Send HTTP request
- 3 Download HTML
- 4 Parse HTML
- 5 Extract required data
- 6 Clean the data
- 7 Store in CSV / DB
- 8 Analyze data
- 9 Feature extraction
- 10 Apply algorithms



```
<html>
<body>
  <h1>iPhone 15</h1>
  <span class="price">₹79,000</span>
  <p class="rating">4.5 (2740)</p>
  <p class="availability">In Stock</p>
</body>
</html>
```



Structured tags such as `<table>`, `<tr>`, `<td>`, headings, and spans often help identify the data to scrape.